

Object-Oriented Programming report

Zhouyue Qian 3220104074 *

(Information and Computational Science Class 2201), Zhejiang University

Due time: December 25, 2024

Abstract

This is an Object-Oriented Programming (OOP) project assignment. In this assignment, I accomplished the following tasks:

1. Implemented my own MyAllocate to replace `$stl::allocate$`.
2. Utilized a memory pool to implement a simple memory pool.

The project's code is located in the sr folder, which includes `$buffer.h$`, `$memory_pool.h$`, `$allocate.h$`.

1 Project Overview

This project implements a custom memory allocator MyAllocate, which is based on memory pool (MemoryPool) technology, aiming to optimize the performance of memory allocation and deallocation. MyAllocate is a template class that supports memory allocation and deallocation operations for any type. It implements the interface of the C++ Standard Library allocator, thus it can be seamlessly integrated with standard library containers such as `std::vector` and `std::list`.

2 Features

- Custom Memory Allocation:
 - Uses memory pool technology to optimize the performance of memory allocation and deallocation.
 - Supports memory allocation for any type (via template parameter T).
- Standard Library Compatibility:
 - Implements the interface of the C++ Standard Library allocator, including `allocate`, `deallocate`, `construct`, and `destroy` methods.
 - Supports seamless integration with `std::vector`, `std::list`, and other standard library containers.
- Memory Pool Management:
 - Uses the MemoryPool class to manage memory blocks, avoiding frequent calls to system-level `malloc` and `free`.

*Electronic address: 3220104074@edu.zju.cn

- The implementation of the memory pool ensures efficient memory allocation and deallocation.
- Type Safety:
 - Ensures type safety for memory allocation and deallocation through the template parameter T.
- Extensibility:
 - Supports allocator copying and rebinding operations, allowing allocators to be shared among different types.

3 Code Structure

3.1 allocator.h

allocator.h is the core file of this project, containing the definition and implementation of the MyAllocate class. Here are the main parts:

- Type Definitions:
 - value_type, pointer, const_pointer, reference, const_reference, size_type, and difference_type.
- Constructors and Destructors:
 - Supports default construction, copy construction, and rebind construction.
 - The destructor is responsible for releasing dynamically allocated resources.
- Memory Allocation and Deallocation:
 - allocate(size_type n, const void* hint = 0): Allocates memory for n objects.
 - deallocate(pointer p, size_type n): Deallocates the memory pointed to by p.
- Object Construction and Destruction:
 - construct(pointer ptr, const value_type& value): Constructs an object at the specified location.
 - destroy(pointer ptr): Destroys the object at the specified location.
- Address Operations:
 - address(reference x) and address(reference x) const: Returns the address of the object.
- Maximum Allocation Size:
 - max_size(): Returns the maximum number of objects that the allocator can allocate.
- New Element Operations:
 - newElement(const value_type& value): Allocates memory and constructs an object.
 - deleteElement(pointer p): Destroys the object and deallocates the memory.

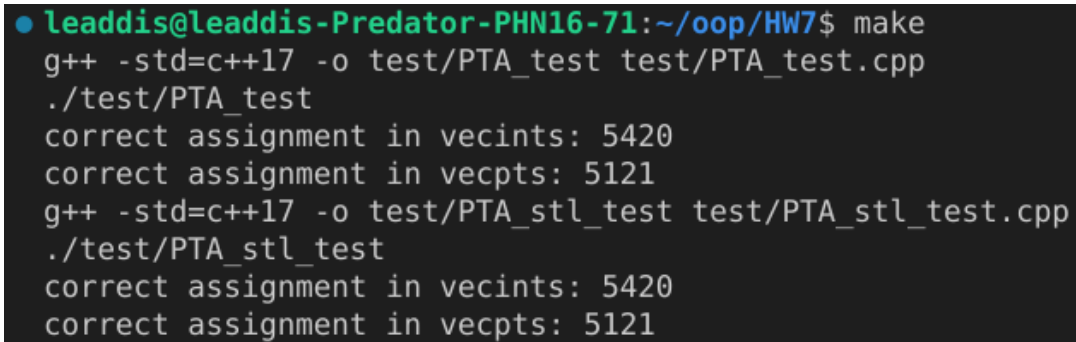
3.2 memory_pool.h

memory_pool.h is the implementation file for the memory pool, responsible for managing the allocation and deallocation of memory blocks. Here are the main parts:

- Memory Allocation:
 - allocate(size_t n): Allocates memory for n objects.
- Memory Deallocation:
 - deallocate(T* p): Deallocates the memory pointed to by p.

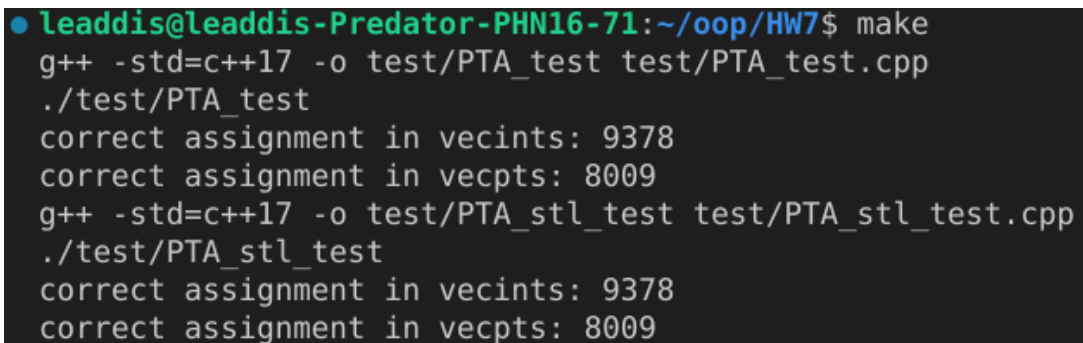
Results

The following results were obtained using the same random number seed.



```
● leaddis@leaddis-Predator-PHN16-71:~/oop/HW7$ make
g++ -std=c++17 -o test/PTA_test test/PTA_test.cpp
./test/PTA_test
correct assignment in vecints: 5420
correct assignment in vecpts: 5121
g++ -std=c++17 -o test/PTA_stl_test test/PTA_stl_test.cpp
./test/PTA_stl_test
correct assignment in vecints: 5420
correct assignment in vecpts: 5121
```

Figure 1: Result Display 1



```
● leaddis@leaddis-Predator-PHN16-71:~/oop/HW7$ make
g++ -std=c++17 -o test/PTA_test test/PTA_test.cpp
./test/PTA_test
correct assignment in vecints: 9378
correct assignment in vecpts: 8009
g++ -std=c++17 -o test/PTA_stl_test test/PTA_stl_test.cpp
./test/PTA_stl_test
correct assignment in vecints: 9378
correct assignment in vecpts: 8009
```

Figure 2: Result Display 2